COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Jul 16, 2026	**DATE SUBMITTED:**	**Jul 20, 2026**
NAME:	Yazinel Corro	**INSTRUCTOR:**	Engr.Jim Lord Quejado

ACTIVITY NO. 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and array

Input:

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
std::string studentName;
int studentAge;
public:
//Constructor
Student(std::string newName = "John Doe", int newAge = 18) {
studentName = newName;
studentAge = newAge;
std::cout << "Constructor was called" << std::endl;
}
//Destructor
~Student(){
std::cout <<"Destructor was called" <<std::endl;
}
//Copy Constructor
Student (const Student &copyStudent) {
    std::cout <<"Copy Constructor Called" <<std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails (){
    std::cout << this ->studentName << " " << this ->studentAge << std::endl;
}

};
int main () {
    Student student1 ("Roman", 28);
    Student student2 (student1);
    Student student3;
    student3 = student2;


    return 0;
}
```

Output:

```
Constructor was called
Copy Constructor Called
Constructor was called
Destructor was called
Destructor was called
Destructor was called
```

Observation:

- The program first reads the main function, In the stack it shows that, Student 1 to Student 3 are already stored. Next, it reads the Information of Student 1 and Student 3. And next is it shows that constructor and copy constructor were called, followed by the pointer to Student and the pointer to the const struct of Student 2. And finally, the destructor was called.

Table 2-1. Initial Driver Program

Input:

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
std::string studentName;
int studentAge;
public:
//Constructor
Student(std::string newName = "John Doe", int newAge = 18) {
studentName = newName;
studentAge = newAge;
std::cout << "Constructor was called" << std::endl;
}
//Destructor
~Student(){
std::cout <<"Destructor was called" <<std::endl;
}
//Copy Constructor
Student (const Student &copyStudent) {
    std::cout <<"Copy Constructor Called" <<std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}
//Display Attributes
void printDetails (){
    std::cout << this ->studentName << " " << this ->studentAge << std::endl;
}

};
int main() {
const size_t j = 5;
Student studentList[j] = {};
std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
int ageList[j] = {15, 16, 18, 19, 16};

return 0;
}
```

Output:

```
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Constructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
Destructor was called
```

Observation:

- First, the program reads the main function. In the stack, the studentlist, namelist, and agelist are already created but they do not contain any information yet. Next, it reads the size, which is 5. Then, it reads the first student's name and age, and the constructor is called, as shown in the output. For the second student, it again reads the student's name and age, and the second constructor is called. The same process happens for the third student, where the name and age are read, and the third constructor is called. As each student's information is entered, the names are stored in namelist and the ages are stored in agelist. The fourth and fifth students follow the same process, with the fourth and fifth constructors being called. After all the students' information has been processed and stored in namelist and agelist, the destructors are called one by one until the fifth destructor is displayed in the output.

Table 2-2. Modified Driver Program with Student Lists

Loop A:

```
for (int i = 0; i < j; i++) {
    Student *ptr = new Student(namesList[i], ageList[i]);
    studentList[i] = *ptr;
    delete ptr;
}
```

Observation:

- Loop A creates a new Student object dynamically for each name and age stored in the arrays. Each object is copied into the studentList array, allowing all student information to be stored correctly. During this process, the constructor and copy assignment operator are called

Loop B:

```
    for (int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    return 0;
}
```

Observation:

- Loop B goes through each object stored in the studentList array and calls the printDetails function. It displays the name and age of every student, confirming that the objects were successfully created and stored in the array.

Output:

```
Constructor was called.
Constructor was called.
Constructor was called.
Constructor was called.
Constructor was called.
Constructor was called.
Copy Assignment Operator Called.
Destructor was called.
Constructor was called.
Copy Assignment Operator Called.
Destructor was called.
Constructor was called.
Copy Assignment Operator Called.
Destructor was called.
Constructor was called.
Copy Assignment Operator Called.
Destructor was called.
Constructor was called.
Copy Assignment Operator Called.
Destructor was called.
```

```
Carly - 15
Freddy - 16
Sam - 18
Zack - 19
Cody - 16
Destructor was called.
Destructor was called.
Destructor was called.
Destructor was called.
Destructor was called.
```

Observation:

- As you can see from the output, the constructors are first called while creating the Student objects. Then the copy assignment operator is called as the dynamically created objects are copied into the studentList array. After that, the program correctly shows the names and ages of all five students. Finally when the program ends the destructors are called which means the objects are properly destroyed. The output in general confirms that the program successfully demonstrates object creation, copying, displaying data, and object destruction.

Modification: No major modifications were made.

Observation: The program executed successfully, displayed the correct student information, and showed that the constructor, copy assignment operator, and destructor worked as expected

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Jenna's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss the

```cpp
#include <iostream>
#include <string>

class Fruit {
private:
    std::string name;
    double price;
    int quantity;

public:
    // Constructor
    Fruit(std::string n = "", double p = 0, int q = 0) :
        name(std::move(n)), price(p), quantity(q) {
        std::cout << "Fruit Constructor Called." << std::endl; }
    // Destructor
    ~Fruit() { std::cout << "Fruit Destructor Called." << std::endl; }
    // Copy Constructor
    Fruit(const Fruit &c) : name(c.name), price(c.price), quantity(c.quantity)
        { std::cout << "Fruit Copy Constructor Called." << std::endl; }
    // Copy Assignment Operator
    Fruit& operator=(const Fruit &c) { std::cout << "Fruit Copy Assignment Called." << std::endl;
        if (this != &c)
        { name = c.name; price = c.price; quantity = c.quantity; } return *this; }
    // Calculate sum
    double calculateSum() { return this->price * this->quantity; }
    // Details
    void printDetails() {
        std::cout << "\nFruit: " << name;
        std::cout << "\nPrice: PHP " << price;
        std::cout << "\nQuantity: " << quantity;
        std::cout << "\nTotal: PHP " << calculateSum() << "\n";
    }
};

```

```
34
35  class Vegetable {
36  private:
37      std::string name;
38      double price;
39      int quantity;
40
41  public:
42      Vegetable(std::string n = "", double p = 0, int q = 0) : name(std::move(n)), price(p), qua
43          { std::cout << "Vegetable Constructor Called." << std::endl; }
44
45      ~Vegetable() { std::cout << "Vegetable Destructor Called." << std::endl; }
46
47      Vegetable(const Vegetable &c) : name(c.name), price(c.price), quantity(c.quantity)
48          { std::cout << "Vegetable Copy Constructor Called." << std::endl; }
49
50      Vegetable& operator=(const Vegetable &c) { std::cout << "Vegetable Copy Assignment Called."
51          if (this != &c)
52          { name = c.name; price = c.price; quantity = c.quantity; } return *this; }
53
54      double calculateSum() { return this->price * this->quantity; }
55
56      void printDetails() {
57          std::cout << "\nVegetable: " << name;
58          std::cout << "\nPrice: PHP " << price;
59          std::cout << "\nQuantity: " << quantity;
60          std::cout << "\nTotal: PHP " << calculateSum() << "\n";
61      }
62  };
```

```
64  //total sum
65  double TotalSum(Fruit* f[], int fSize, Vegetable* v[], int vSize) {
66      double total = 0;
67      for (int i = 0; i < fSize; i++) if (f[i]) total += f[i]->calculateSum();
68      for (int i = 0; i < vSize; i++) if (v[i]) total += v[i]->calculateSum();
69      return total;
70  }
71  //main driver
72  int main() {
73
74      Fruit* fruitList[2] = { new Fruit("Apple", 10, 7), new Fruit("Banana", 10, 8) };
75      Vegetable* vegList[2] = { new Vegetable("Broccoli", 60, 12), new Vegetable("Lettuce", 50, 10)
76
77      std::cout << "\n--- Jenna's Grocery List ---" << std::endl;
78      for (int i = 0; i < 2; i++) if (fruitList[i]) fruitList[i]->printDetails();
79      for (int i = 0; i < 2; i++) if (vegList[i]) vegList[i]->printDetails();
80
81      // total Sum of all objects listed in Jenna's Grocery List
82      std::cout << "\nTotal Sum: PHP " << TotalSum(fruitList, 2, vegList, 2) << std::endl;
83
84      //Deleting Lettuce
85      std::cout << "\n--- Deleting Lettuce ---" << std::endl;
86      delete vegList[1];
87      vegList[1] = nullptr;
88
89      std::cout << "Updated Total Sum: PHP " << TotalSum(fruitList, 2, vegList, 2) << std::endl
90
91      // Memory Cleanup
92      for (int i = 0; i < 2; i++) delete fruitList[i];
93      for (int i = 0; i < 2; i++) delete vegList[i];
94
95      return 0;
```

```
"C:\Users\yazinel\OneDrive\Desktop\cpe 010 - hoa 2\cmake-build-debug\hoa2_1.exe"
Fruit Constructor Called.
Fruit Constructor Called.
Vegetable Constructor Called.
Vegetable Constructor Called.

--- Jenna's Grocery List ---

Fruit: Apple
Price: PHP 10
Quantity: 7
Total: PHP 70

Fruit: Banana
Price: PHP 10
Quantity: 8
Total: PHP 80

Vegetable: Broccoli
Price: PHP 60
Quantity: 12
Total: PHP 720

Vegetable: Lettuce
Price: PHP 50
Quantity: 10
Total: PHP 500
```

```
Vegetable: Lettuce
Price: PHP 50
Quantity: 10
Total: PHP 500

Total Sum: PHP 1370

--- Deleting Lettuce ---
Vegetable Destructor Called.
Updated Total Sum: PHP 870

Fruit Destructor Called.
Fruit Destructor Called.
Vegetable Destructor Called.
```

Summary of lessons learned :

- I learned the difference between static and dynamic memory allocation, and how

arrays and pointers work in C++. I also learned the importance of constructors, destructors, copy constructor and copy assignment operators to manage objects and memory.
Analysis of the procedure: - The procedure helped me learn how to create objects using static and dynamic memory allocation. It also showed how to use loops to initialize, store and display objects in an array and how to use the Rule of Three in a class.
Analysis of the supplementary activity: - The supplementary activity allowed me to apply what I learned by creating classes, using dynamic memory allocation, and calculating the total price of grocery items.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement? - I was able to finish the activity and understand most of the concepts, although I was a bit confused at first about dynamic memory allocation and the Rule of Three. With practice I can get better understanding of pointers, copy constructors and memory management in C++.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT and Quillbot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code Assistance and Grammar Correction

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I asked if my code was correct since some part of code got me a confused.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.